

System Programming Portfolio

Scheduler & IPC Projects — Technical Reference

A technical reference covering CPU scheduling algorithm design, inter-process communication mechanisms, synchronization strategy, and build configuration for the accompanying C implementation.

Document	DOCUMENTATION.pdf
Companion File	README.md
Language / Platform	C (GCC) — Linux
Version	1.0

Technical Documentation — System Programming Portfolio

This document provides an in-depth technical reference for the two modules in this repository: the **CPU Scheduler Simulator** and the **IPC Projects Suite**. It covers algorithm logic, data structures, function-level responsibilities, synchronization design, and compilation details.

For a quick project overview, see [README.md](#)

Table of Contents

1. Scheduler Module
 - 1.1 Data Structures
 - 1.2 FCFS — First Come First Serve
 - 1.3 SJF — Shortest Job First
 - 1.4 Priority Scheduling (Preemptive)
 - 1.5 Round Robin
 - 1.6 utils.c / utils.h
2. IPC Module
 - 2.1 Shared Memory Chat
 - 2.2 Producer-Consumer
 - 2.3 Semaphore + Shared Memory Combo
 - 2.4 Dining Philosophers
3. Compilation & Linking Notes
4. Testing Guidance
5. Known Limitations
6. Glossary

1. Scheduler Module

The scheduler module simulates how an operating system selects which process runs next on the CPU. Each algorithm is implemented as a separate translation unit and driven by a common `scheduler.c` entry point.

1.1 Data Structures

All algorithms operate on a common `Process` structure (defined in `utils.h`), typically containing:

```
typedef struct {
    int pid;           // Process ID
    int arrival_time;  // Time process enters the ready queue
    int burst_time;    // Total CPU time required
    int remaining_time; // Used by preemptive algorithms
    int priority;      // Used by priority scheduling
    int completion_time;
    int waiting_time;
    int turnaround_time;
} Process;
```

If your actual struct differs, update this section to match your `utils.h` definitions exactly — this documentation should be kept in sync with the source.

Derived metrics (computed after scheduling):

- `turnaround_time = completion_time - arrival_time`
- `waiting_time = turnaround_time - burst_time`

1.2 FCFS — First Come First Serve

File: `fcfs.c`

- **Type:** Non-preemptive
- **Logic:** Processes are sorted by arrival time and executed strictly in that order. Once a process starts, it runs to completion.
- **Time Complexity:** $O(n \log n)$ for sorting, $O(n)$ for scheduling.
- **Fairness:** Simple but prone to the **convoy effect** — short processes can be stuck waiting behind a long process.

1.3 SJF — Shortest Job First

File: `sjf.c`

- **Type:** Non-preemptive
- **Logic:** Among all processes that have arrived, the one with the shortest burst time is selected next.
- **Time Complexity:** $O(n^2)$ in the naive implementation (linear scan per selection); can be improved with a min-heap to $O(n \log n)$.
- **Trade-off:** Minimizes average waiting time but can cause **starvation** of longer processes.

1.4 Priority Scheduling (Preemptive)

File: `priority.c`

- **Type:** Preemptive
- **Logic:** At every time unit, the ready process with the highest priority (lowest numeric value, by convention) is given the CPU. If a higher-priority process arrives mid-execution, the current process is preempted.
- **Time Complexity:** $O(n \times \text{max_burst})$ in a time-unit simulation approach.
- **Trade-off:** Can cause starvation of low-priority processes; typically paired with **aging** in production schedulers (not implemented here).

1.5 Round Robin

File: `round robin.c`

- **Type:** Preemptive
- **Logic:** Each process is given a fixed **time quantum**. If it doesn't finish within that quantum, it's placed at the back of the ready queue and the next process runs.
- **Time Complexity:** $O(n \times (\text{burst_time} / \text{quantum}))$ in the simulation loop.
- **Trade-off:** Fair and responsive, but throughput depends heavily on quantum size — too small increases context-switch overhead, too large degrades to FCFS behavior.

1.6 `utils.c` / `utils.h`

Shared helper functions used across all scheduling algorithms, typically including:

- Input collection (number of processes, arrival/burst/priority)
- Sorting utilities (e.g., sort by arrival time)
- Output formatting (results table, per-process and average metrics)

2. IPC Module

The IPC module demonstrates classic synchronization problems using POSIX shared memory, semaphores, and threads.

2.1 Shared Memory Chat

File: `shared_memory_chat.c`

- **Mechanism:** `shmget`, `shmat`, `shmdt`, `shmctl`
- **Design:** A shared memory segment acts as a message buffer between two independent processes. One process writes a message into the segment; the other reads it.
- **Synchronization concern:** Without additional coordination (e.g., semaphores or a simple flag/turn variable), this demo is subject to race conditions if both processes access the segment simultaneously — worth noting explicitly if this example doesn't already use a semaphore for turn-taking.
- **Cleanup:** Shared memory segments persist in the kernel until explicitly removed with `shmctl(id, IPC_RMID, NULL)` — omitting this will leak memory segments visible via `ipcs -m`.

2.2 Producer-Consumer

File: `producer_consumer.c`

- **Mechanism:** `pthread`, `semaphore.h` (POSIX semaphores)
- **Design:** A bounded circular buffer is shared between producer and consumer threads.
 - `sem_empty` — counts available empty slots
 - `sem_full` — counts filled slots
 - A mutex (or binary semaphore) protects the critical section around buffer indices
- **Classic risk:** Deadlock if `wait()` / `post()` calls are mismatched, or a busy-wait if semaphores are replaced with naive polling.

2.3 Semaphore + Shared Memory Combo

File: `shm_semaphore_combo.c`

- **Mechanism:** Combines `sys/shm.h` for data storage with named or unnamed semaphores (`sem_open` / `sem_init`) for access control.

- **Design:** Demonstrates the realistic pattern used in production IPC — shared memory alone provides no synchronization guarantees, so a semaphore gates read/write access to prevent torn reads or lost updates.

2.4 Dining Philosophers

File: `dining_philosophers.c`

- **Mechanism:** `pthread`, array of semaphores (one per fork/chopstick)
- **Design:** N philosophers alternate between thinking and eating; each needs two adjacent forks to eat.
- **Deadlock prevention strategy:** Typically one of:
 - Asymmetric ordering (one philosopher picks up the left fork first, another picks up the right — breaking the circular wait condition)
 - Resource hierarchy (always acquire the lower-numbered fork first)
 - Limiting the number of philosophers allowed to eat concurrently to N-1

3. Compilation & Linking Notes

Requirement	Flag
POSIX threads	<code>-lpthread</code>
POSIX semaphores (some libc versions)	<code>-lrt</code>
Debug symbols	<code>-g</code>
Warnings (recommended)	<code>-Wall -Wextra</code>

Example manual compilation (if not using `make`):

```
gcc -Wall -Wextra -pthread -o producer_consumer producer_consumer.c
gcc -Wall -Wextra -o shared_memory_chat shared_memory_chat.c
```

4. Testing Guidance

Since these are OS-level concurrency demos, correctness isn't just "does it compile" — race conditions and deadlocks may not appear on every run. Suggested checks:

- **Scheduler:** Verify computed waiting/turnaround times by hand against a Gantt chart for small inputs (3-5 processes).

- **Producer-Consumer:** Run repeatedly with a small buffer size and high thread iteration counts to surface race conditions.
- **Dining Philosophers:** Run with logging of fork acquisition order to confirm no circular-wait pattern emerges under load.
- **Shared Memory Chat:** Confirm the segment is removed after each run (`ipcs -m` should show no leftover segments).

5. Known Limitations

- Scheduler currently assumes all input is well-formed (no negative burst times, no malformed input handling).
- IPC examples are for demonstration/learning and are not hardened for production use (no robust error handling on `shmget` / `sem_init` failures in all cases).
- No automated test suite is currently included (see Roadmap in `README.md`).

6. Glossary

Term	Meaning
Burst Time	Total CPU time a process needs to complete
Turnaround Time	Total time from arrival to completion
Waiting Time	Time spent in the ready queue, not executing
Preemption	Interrupting a running process to run another
Race Condition	Undefined behavior from unsynchronized concurrent access to shared data
Deadlock	A state where processes/threads wait on each other indefinitely, none able to proceed
Semaphore	A synchronization primitive that controls access to a shared resource via a counter
